

Situation-Aware Integrated Sensing and Communication for 6G Smart Warehouses: A Sionna RT and ns-3 Co-Simulation Framework

Aung Kaung MYAT, Hrishikesh Dutta, Roberto Minerva, Noel Crespi

SAMOVAR, Télécom SudParis

Institut Polytechnique de Paris

Palaiseau, France

Email: {aung-kaung.myat, hrishikesh.dutta, roberto.minerva, noel.crespi}@telecom-sudparis.eu

Abstract—Smart warehouses need wireless networks that can support sensors, controllers, video traffic, and mobile robots while also reacting to changes in the environment. This paper presents a situation-aware Integrated Sensing and Communication (ISAC) co-simulation framework that connects Sionna RT ray tracing with ns-3/5G LENA networking. The framework uses ray-traced multipath to detect reflected targets, filters clutter, tracks moving objects, and then uses the confirmed target positions to form sensing-informed beams.

The framework is evaluated in two stages. First, controlled free-space and warehouse scenes are used to test the sensing and beamforming pipeline under clear and cluttered conditions. Second, an end-to-end warehouse Digital Twin is used to study the same mechanism with mobile robots, package and rack sensors, cameras, video clients, and mixed application traffic. The controlled results show that ISAC reduces path loss in both scenes, that a wider beam gives the best communication behavior, and that tracking is needed to suppress false detections. Deeper propagation gives a small detection gain, but it can also increase clutter in the warehouse. In the full warehouse use case, the 4×4 gNB array gives the best balance: the 2×2 array is not stable enough, while the 8×8 array increases power consumption without a proportional performance gain. These results show that practical ISAC evaluation must consider sensing quality, communication performance, and energy cost together.

Index Terms—Integrated sensing and communication (ISAC), 6G, situation awareness, ray tracing, Sionna RT, ns-3, 5G LENA, adaptive beamforming, multi-target tracking, Digital Twin, smart warehouse.

I. FRAMEWORK AND METHODOLOGY

In this section, we present the proposed situation-aware ISAC framework which is implemented in ns-3 with Sionna RT, which can be used to simulate the ISAC pipeline for 6G networks. The overall system architecture is presented first, followed by the detailed description of the ISAC pipeline modules.

A. ISAC Pipeline System Architecture

The proposed Integrated Sensing and Communication (ISAC) pipeline system architecture, presented in Figure 1, is designed to seamlessly integrate environmental sensing and adaptive beamforming for enabling dynamic, direction-aware communication into a cohesive framework. There are four main modules in the ISAC pipeline: the Sensing Module, the Multi-Target Tracking Module, the Adaptive Beamforming Module, and the Communication Module.

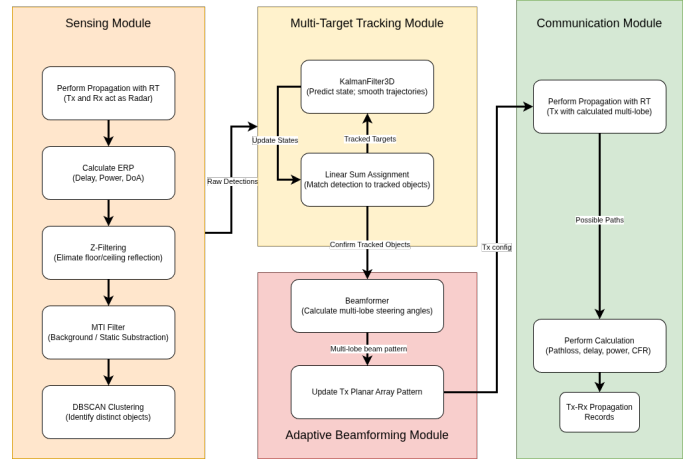


Fig. 1. ISAC Pipeline System Architecture.

The **Sensing Module** is responsible for capturing and processing the radio propagation signals from the environment, to detect the objects in the environment and estimate their states, including position, and velocity. In order to achieve this, the sensing module use the MIMO antennas as the monostatic radar antennas to transmit and receive the radio signals. It begins by emitting rays from the transmitter (Tx) and the receiver (Rx) captures the reflected rays from the environment. From the received reflected rays, it extract essential information such as delay, power, Direction of Arrival (DoA), etc and it calculate the 3D Equivalent Reflection Point (ERP), estimating the exact physical origin of the bounce rays. To eliminate the unwanted reflections from the structural noises from the environment such as floor and ceiling, a spatial filter is applied to filter out rays that originated outside the a defined vertical region of interest and this process is called Z-filtering. After rays from the floor and ceiling are filtered out, the Moving Target Indicator (MTI) Filter is applied to filter out the rays reflected from the static objects in the environment such as pillars and walls. Finally, the Density-Based Spatial Clustering of Applications with Noise (DBSCAN) algorithm is applied on the remaining rays in order to identify and group the rays into clusters that correspond to distinct objects in the environment. The centroid of each cluster is calculated and used as the estimated position of the object (i.e., raw detection).

These raw detections are then passed to the **Multi-Target**

Tracking Module, which is responsible for monitoring the dynamic states of moving objects over time. It utilizes a Linear Sum Assignment algorithm to optimally match incoming raw detections with previously tracked objects that are kept in the memory. Matched detections are then passed to a 3D Kalman Filter, which predict the future state (e.g. position) of objects and make the transition of movement along their trajectories smooth. This recursive loop continuously updates the tracked states of the objects, resulting in a refined set of confirmed tracked objects.

When the moving objects in the environment are detected and tracked successfully, the information of the detected objects can be used to form beams towards the detected objects to improve the quality of the communication link. Based on the estimated position, velocity, and direction of the detected objects, the Beamformer of the **Adaptive Beamforming Module** calculate the multi-lobe steering angles required to directed beams towards the identified objects. The multi-lobe beam pattern calculated is then used to update the Tx array configuration to focus the beams towards the detected objects.

Finally, the **Communication Module** calculate propagation in the channel with the calculated multi-lobe beam pattern. The module calculate propagation metrics such as pathloss, delay, received power, and Channel Frequency Response (CFR) based on the multi-lobe beam pattern. These extracted propagation related information are then forwarded to ns-3 simulator to perform the communication simulation.

B. Ray-tracing Based Sensing Module

Path solver in Sionna RT provides the path information between the transmitter and receiver based on the 3D scene environment. It include propagation delay, channel coefficients, angle of departure (AoD), angle of arrival (AoA), and doppler shift.

1) *Pre-processing*: To get a single representative value for each path, we compute the average path power, delay, angles of arrival, and angles of departure. For each path ℓ , the average path power is computed from channel coefficients, average delay from the propagation delay, average angles of arrival are computed.

$$\overline{P}_\ell = \frac{1}{|I|} \sum_{i \in I} |a_i^2|, \quad \overline{\tau}_\ell = \frac{1}{|I|} \sum_{i \in I} |\tau_{\ell,i}| \quad (1)$$

$$\overline{\theta}_\ell = \frac{1}{|I|} \sum_{i \in I} \theta_{\ell,i}, \quad \overline{\phi}_\ell = \frac{1}{|I|} \sum_{i \in I} \phi_{\ell,i} \quad (2)$$

Paths with $\overline{\tau}_\ell \leq 10^{-9}$ s are dropped to remove paths with almost zero delays to keep only paths that likely traveled a meaningful distance. Then, the distance or range for each path is computed by converting remaining path delays using the speed of light.

$$L_\ell = c\overline{\tau}_\ell \quad (3)$$

Instead of a fixed limit, we use a power threshold that adjusts based on distance so we do not lose weak signals from far away.

$$s_\ell = \max\left(\frac{L_\ell}{10}, 1\right)^{1.1}, \quad P_{\text{th},\ell} = \frac{P_{\min}}{s_\ell}, \quad (4)$$

Paths with $\overline{P}_\ell > P_{\text{th},\ell}$ are kept. Finally, each remaining angle pair is converted to a 3D unit direction,

$$\mathbf{d}_\ell = [\sin \theta_\ell \cos \phi_\ell, \sin \theta_\ell \sin \phi_\ell, \cos \theta_\ell]^\top \quad (5)$$

2) *Equivalent Reflection Point (ERP)*: In complex environments, an electromagnetic wave often undergoes multiple interactions such as specular reflections, edge diffractions, or diffuse scattering before returning to the receiver. Tracing every interaction sequence requires prior knowledge of the object's exact geometry and material properties. Instead, the core idea behind detection in this framework is to abstract these potentially complex multipath trajectories into an Equivalent Reflection Point (ERP). The ERP combines the total delay and direction change of the entire signal path into a single 3D point, removing the need to find every individual bounce along the way.

In our proposed framework, a monostatic ISAC setup is used where the transmitter and receivers are collocated. Each remaining path ℓ is mapped to a single ERP in 3D space:

$$\mathbf{p}_\ell^{\text{ERP}} = \mathbf{p}_{\text{gNB}} + \left(\frac{L_\ell}{2} + \delta\right) \mathbf{d}_\ell, \quad (6)$$

, where $\mathbf{p}_{\text{gNB}}$ is the gNB spatial location, and $\mathbf{d}_\ell$ is the unit direction vector of the path. To account for the round-trip distance in monostatic case, we divide the path delay by $2(L_\ell/2)$. δ , a small calibration offset is added to mitigates the bias introduced by reflection on an object's exterior surface rather than precisely at its geometric centre. By transforming raw angle-delay measurements into 3D coordinates, the ERP allows to reconstruct the target's shape and locate its position.

3) *Z-filtering*: Rays reflected from the floor and ceiling of the environment create ghost ERPs that lie outside the height range of targets. To reduce these ghost ERPs, a simple filtering is applied.

$$z_{\min} \leq z_\ell^{\text{ERP}} \leq z_{\max} \quad (7)$$

, where $z_{\min}$ and $z_{\max}$ are the minimum and maximum height of the target object that we would like to detect.

4) *Moving-Target-Indication (MTI) Filter*: To distinguish the static background and moving object, a Moving-Target-Indication (MTI) filter is used. The first ERP frame in the simulation is considered as a static background point cloud

$$\mathcal{B} = \{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_M\} \quad (8)$$

where $\mathbf{b}_i$ is the i -th ERP in the first frame. For all the ERP that follows first frame, the nearest-background distance which is implemented using k-d tree:

$$d_{\mathcal{B}}(\mathbf{p}) = \min_{\mathbf{b} \in \mathcal{B}} \|\mathbf{p} - \mathbf{b}\|_2 \quad (9)$$

is computed and only points that satisfy $d_{\mathcal{B}}(\mathbf{p}) > d_{\text{MTI}}$ are kept.

5) *DBSCAN Clustering*: When rays of signal strike on a single physical object, typically, multiple rays will be reflected from different parts of the object. This results in multiple ERPs generated from the same object. To identify and isolate each individual objects, in the environment, clustering algorithms is applied on ERP point cloud which group several points into clusters that corresponding to actual objects. In this framework

DBSCAN clustering algorithm is used as it does not require the number of clusters (i.e. number of objects) in advance, filters out isolated noise points and good at identifying dense regions.

For each identified cluster $\mathcal{C}_k$, the representative point of each cluster $\mathbf{z}_k$ is selected by choosing the point which has the highest received signal power:

$$\mathbf{z}_k = \mathbf{p}_{n^*}, \quad \text{where } n^* = \arg \max_{n \in \mathcal{C}_k} \overline{P}_n \quad (10)$$

, and overall received signal power of the cluster P_k is computed by summing the powers of all the ERPs in the cluster:

$$P_k = \sum_{n \in \mathcal{C}_k} \overline{P}_n \quad (11)$$

To avoid duplicated reporting of the same physical object, the candidate clusters are sorted in descending order of their order of their total power P_k . If the Euclidean distance between the representative points of any two clusters is less than the separation threshold:

$$\|\mathbf{z}_i - \mathbf{z}_j\|_2 < d_{\text{sep}} \quad (12)$$

the cluster with lower total power is removed. Finally, it returns at most N_{max} candidates with strongest received signal power are returned as tuples:

$$\mathcal{D}_k = (k, \mathbf{z}_k, P_k) \quad (13)$$

, where k is the cluster index, $\mathbf{z}_k$ is the representative point of the cluster, and P_k is the total received signal power of the cluster.

C. Multi-Target Tracking Module

At each sensing update t , the sensing module produces a set of candidates for the detected objects, each with a 3D representative position and an accumulated power. However, when there is occlusion or clutter, the sensing module may produce ghost detections or miss detections. To address this issue and achieve temporal consistency, the multi-target tracking module converts these detections into temporally consistent tracked targets. Then, these traced targets are passed to the beamforming module.

Let the detection set at sensing update t be

$$\mathcal{D}_t = \{(\mathbf{z}_{t,j}, P_{t,j})\}_{j=1}^{N_t} \quad (14)$$

where $\mathbf{z}_{t,j} \in \mathbb{R}^3$ is the representative position of detection j and $P_{t,j}$ is its total power. The sensing module uses power to rank candidate detections, while the tracking module uses position to match detections over time.

Each active tracked target is represented by a constant-velocity Kalman filter with state

$$\mathbf{x}_{t,i} = [x_{t,i}, y_{t,i}, z_{t,i}, v_{x,t,i}, v_{y,t,i}, v_{z,t,i}]^\top. \quad (15)$$

, where the first three components represent the positions and the last three components represent the velocities of the i -th tracked target.

At each sensing update t , the next possible state of each active tracked target is predicted as

$$\mathbf{x}_{t|t-1,i} = F\mathbf{x}_{t-1|t-1,i}, \quad (16)$$

$$P_{t|t-1,i} = F P_{t-1|t-1,i} F^\top + Q, \quad (17)$$

where Q is the process-noise covariance and

$$F = \begin{bmatrix} I_3 & \Delta t I_3 \\ \mathbf{0}_{3 \times 3} & I_3 \end{bmatrix}. \quad (18)$$

Δt corresponds to one sensing update.

After prediction, the tracking algorithm matches the predicted tracked targets to the detections in the current sensing update. This matching process is known as data association in multi-target tracking. For each predicted tracked target i and detection j , the matching cost is the Euclidean distance

$$c_{ij} = \|H\mathbf{x}_{t|t-1,i} - \mathbf{z}_{t,j}\|_2, \quad H = [I_3 \quad \mathbf{0}_{3 \times 3}]. \quad (19)$$

Pairs with $c_{ij} > c_{\text{max}}$ are considered unlikely and are assigned a large penalty. The Hungarian algorithm [9] then selects the set of matches with the minimum total cost:

$$\pi^* = \arg \min_{\pi} \sum_{(i,j) \in \pi} c_{ij}. \quad (20)$$

This joint matching avoids greedy decisions when multiple targets and detections are close to each other.

Each accepted tracked target-detection pair is corrected using the Kalman update. The measurement residual and Kalman gain are

$$\mathbf{y}_{t,i} = \mathbf{z}_{t,j} - H\mathbf{x}_{t|t-1,i}, \quad (21)$$

$$S_{t,i} = H P_{t|t-1,i} H^\top + R, \quad (22)$$

$$K_{t,i} = P_{t|t-1,i} H^\top S_{t,i}^{-1}, \quad (23)$$

where R is the measurement-noise covariance.

The corrected state and covariance are then

$$\mathbf{x}_{t|t,i} = \mathbf{x}_{t|t-1,i} + K_{t,i} \mathbf{y}_{t,i}, \quad (24)$$

$$P_{t|t,i} = (I - K_{t,i} H) P_{t|t-1,i}. \quad (25)$$

After a successful update, the age of the tracked target is increased and its missed-frame counter is reset.

Unmatched predicted targets are kept temporarily by increasing their missed-frame counter, which allows the tracker to bridge short sensing gaps. They are removed once this counter reaches the maximum allowed value M_{max} . For detections that are not matched to any of the existing tracked targets, a new tracked target is initialized with the detection's position and zero velocity:

$$\mathbf{x}_0 = [z_x, z_y, z_z, 0, 0, 0]^\top. \quad (26)$$

The initial position is also stored as $\mathbf{p}_{\text{start}}$ for later confirmation.

The tracker exports only confirmed moving tracked targets. A tracked target is confirmed when it has survived for at least A_{min} frames and its displacement from the starting position exceeds d_{min} :

$$\|H\mathbf{x}_{t,i} - \mathbf{p}_{\text{start},i}\|_2 > d_{\text{min}}. \quad (27)$$

This rule reduces short-lived detections and static residual clutter. In this work, we set $M_{\max} = 3$, $A_{\min} = 3$, and $d_{\min}$ is configurable at runtime. The output of this tracking module is the list of confirmed 3D target positions used by the sensing-informed beamforming stage.

D. Sensing-Informed Adaptive Beamforming

The sensing-informed beamforming module uses the confirmed 3D target positions from the tracking module to steer the beam towards the moving targets. It is applied only when there is at least one moving target is confirmed. Otherwise, the simulator perform propagation without using the adaptive beamforming pattern. This design keeps the propagation model unchanged when sensing does not provide reliable target information.

Let the confirmed target positions at time t be

$$\mathbf{P}_t = \{\mathbf{p}_{t,m}\}_{m=1}^{M_t}, \quad (28)$$

where $\mathbf{p}_{t,m}$ is the 3D position of tracked target m .

For each target, the module first computes the direction from the gNB position $\mathbf{p}_{\text{gNB}}$ to the target:

$$\mathbf{u}_{t,m} = \mathbf{p}_{t,m} - \mathbf{p}_{\text{gNB}}. \quad (29)$$

This vector is then converted into spherical steering angles:

$$\theta_{t,m} = \arccos\left(\frac{u_{t,m,z}}{\|\mathbf{u}_{t,m}\|_2}\right), \quad (30)$$

$$\phi_{t,m} = \text{atan2}(u_{t,m,y}, u_{t,m,x}), \quad (31)$$

where $\theta_{t,m}$ is the zenith angle and $\phi_{t,m}$ is the azimuth angle.

Instead of computing a separate digital precoder, the implementation constructs a custom Sionna RT antenna pattern with one Gaussian lobe per confirmed target. For an observation direction (θ, ϕ) , let $\hat{\mathbf{r}}(\theta, \phi)$ be the corresponding unit direction vector. The angular separation between this observation direction and the steering direction of target m is

$$\psi_m(\theta, \phi) = \arccos(\hat{\mathbf{r}}(\theta, \phi)^\top \hat{\mathbf{r}}(\theta_{t,m}, \phi_{t,m})). \quad (32)$$

Using a beamwidth parameter $\sigma = \text{deg2rad}(W)$, the multi-lobe gain is

$$G(\theta, \phi) = \sum_{m=1}^{M_t} \exp\left(-\frac{\psi_m(\theta, \phi)^2}{2\sigma^2}\right). \quad (33)$$

The resulting parameters are used to configured the beamforming pattern, which is then updated on the gNB. After the adaptive beamforming pattern is configured, the simulator performs the propagation calculation. If the beamforming update fails or no confirmed target is available, the simulator falls back to the original propagation model. To summarize, the sensing-informed adaptive beamforming module uses the confirmed 3D target positions from the tracking module to steer the beam towards the moving targets and channel response with beamforming is then calculated and returned to ns-3 for further processing.

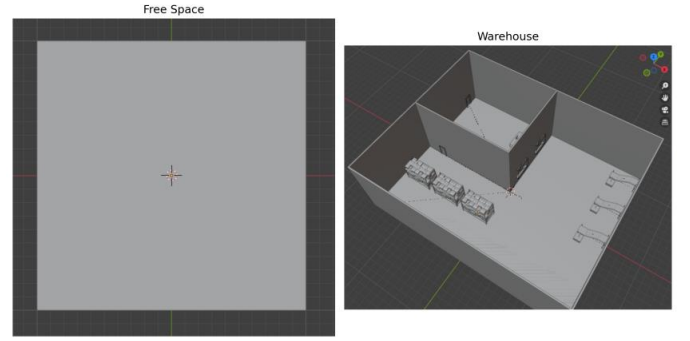


Fig. 2. Controlled-validation scenes.

II. EXPERIMENTAL EVALUATION

In this section, we present the evaluation of the proposed ISAC framework. First, the framework is tested that the sensing, tracking, and sensing-informed beamforming are behaving as expected without the ns-3 traffic or protocol effects. After that, the same framework is evaluated in the warehouse Digital Twin scenario, where the channel, applications, robot mobility, and energy behavior interact in an end-to-end NR simulation. The first part of the experiment evaluated the components of the framework and the second part evaluated the framework's application to the use case.

A. Controlled Validation Results

To validate the framework under reproducible propagation conditions, the first experiment is designed as controlled validation experiment. For this reason, the framework is tested with two different scenes: free space and an indoor warehouse as shown in Figure 2. Free space scene is least obstructed and it is used as a baseline. Warehouse scene represents the indoor use case where there are multiple reflections and blockages from the warehouse infrastructure. These two scenes make it possible to observe how the same sensing and beamforming pipeline changes when visibility and clutter change.

Each scene is tested with the same basic procedure so that the differences in the results are caused by scene and ISAC configuration rather than changes in workflow. The free-space and warehouse use cases use the sinusoidal trajectories of the robots shown in Figure 3. The system is evaluated for 100 time steps with a 0.2 seconds interval, providing a 20 seconds observation window. At each step, propagation is computed and, when situation awareness is enabled, the sensing module produces detections that are filtered, clustered, tracked, and then used for sensing-informed beamforming. The baseline does not use ISAC features and performs communication only, while the ISAC variants enable or disable specific functions such as MTI, tracking, beamwidth selection, and propagation depth in order to isolate one part of the proposed pipeline.

The `isac_full` variant test the whole pipeline. The `isac_no_mti` variant disables only MTI to test whether removing static-reflection suppression affects detection stability. The `isac_narrow` and `isac_wide` variants change only beamwidth to compare tighter target focusing against wider coverage under localization error. The `isac_deep`

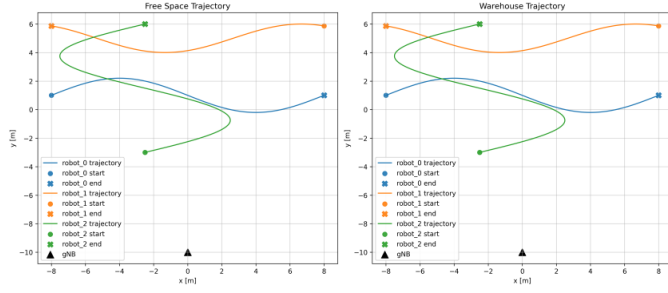


Fig. 3. Deterministic robot trajectories used in the controlled validation scenes.

TABLE I
FREE-SPACE PROPAGATION METRICS.

Variant	Path loss [dB]	Delay [ns]	Rx power [dBm]
baseline	102.39	54.10	-56.39
isac_full	86.30	54.10	-40.30
isac_no_mti	85.53	54.10	-39.53
isac_narrow	86.82	54.10	-40.82
isac_wide	84.39	54.10	-38.39
isac_deep	85.54	54.10	-39.54
isac_no_tracker	85.43	54.10	-39.43

variant increases propagation depth to test whether additional multipath improves sensing or mainly adds clutter. The *isac_no_tracker* variant disables temporal tracking to test whether raw frame-by-frame detections are sufficient for beamforming. After each run, the estimated target positions are matched to the known ground truth from the trajectories with the Hungarian algorithm, where unmatched estimates are considered as false positives and unmatched targets are considered as missed detections.

1) *Free-Space Scene*: The results in Table I shows that enabling ISAC reduces the mean path loss from 102.39 dB to the range 84.39–86.82dB. The mean path delay remains at 54.10 ns as the mean distance is same for all the variants. The differences of performance is more observable in received power as in path loss. Among the tested ablations, *isac_wide* achieves the lowest path loss and the strongest received power, which indicates that the wider beam is beneficial when clutter is minimal.

The sensing and tracking results in Table II follow a similar trend. The tracking-enabled ISAC variants achieve detection rates between 0.847 and 0.903, while their false-positive rates remain close to 0.018–0.019. Among them, *isac_deep* achieves the highest detection rate 0.903 and lowest missed-detection rate 0.097, which shows that additional propagation depth can help to detect better but the gain over the other tracked variants is small. However, the extra computation necessary for deeper propagation does not worth the gain. The *isac_no_tracker* variant achieves the highest raw detection rate (0.970), but it also increases the false-positive rate to 0.176. This means that the tracker is useful because it trades raw sensitivity for cleaner detection.

2) *Warehouse Scene*: In the warehouse scene, because of obstacles such racks and partial visibility create the clutter which makes sensing difficult. The propagation results in Table III show that ISAC still improves communication, but the gain

TABLE II
FREE-SPACE SENSING METRICS.

Variant	Det.	FPR	Miss	Err. [m]	Cont.
baseline	0.000	0.000	1.000	–	0.000
isac_full	0.847	0.019	0.153	1.148	0.550
isac_no_mti	0.897	0.018	0.103	1.125	0.537
isac_narrow	0.897	0.018	0.103	1.125	0.537
isac_wide	0.897	0.018	0.103	1.125	0.537
isac_deep	0.903	0.018	0.097	1.126	0.510
isac_no_tracker	0.970	0.176	0.030	1.208	0.583

TABLE III
WAREHOUSE-SCENE PROPAGATION METRICS.

Variant	Path loss [dB]	Delay [ns]	Rx power [dBm]
baseline	109.16	53.95	-63.16
isac_full	97.90	53.95	-51.90
isac_no_mti	97.39	53.95	-51.39
isac_narrow	101.75	53.95	-55.75
isac_wide	94.50	53.95	-48.50
isac_deep	97.44	53.95	-51.44
isac_no_tracker	98.52	53.95	-52.52

is smaller than free space case. With ISAC, mean path loss drops from 109.16 dB to 94.50–101.75 dB across the ISAC variants. Like in free space case, *isac_wide* is the best in terms of path loss and received power, where it achieves 94.50 dB path loss and -48.50 dBm received power.

The sensing results in Table IV show that sensing in the warehouse scene is more difficult than in free space. The tracked variants of ISAC detect only 0.280–0.290 of the target instances and their missed-detection rates remain high around 0.710–0.720. This low detection rate is due to the racks, walls, and indoor objects block some target reflections. As a result, some of true robot reflections are either too weak, too fragmented, or too close to clutter to survive the complete sensing pipeline and counted as missed detections. The *isac_deep* variant produce much higher false-positive rates, showing that deeper ray exploration adds more clutter and makes it harder to distinguish from the target reflections. Like in free space, the tracking improve the detection rate and reduce the false-positive rate as the false-positive rate of *isac_no_tracker* is much higher than the others with tracking.

This validation confirms two connected observations. First, the framework behaves as expected in free space, where both communication and sensing benefit from favorable visibility. Across both scenes, the wider beamwidth gives the best propagation result because it covers target positions uncertainty better than the narrower beam. Increasing propagation depth improves detection rate slightly, but the gain is very small and, in the warehouse case, it increases false positive rate. Second, the framework works in the warehouse case as well but the performance is degraded due to racks and partial visibility. The no-tracker cases show that raw detections alone are not enough as it is needed to reduce the false-positive rate.

B. Warehouse Use-Case Results

After the controlled validation, the framework proposed is applied to simulate the warehouse digital twin application.

TABLE IV
WAREHOUSE-SCENE SENSING METRICS.

Variant	Det.	FPR	Miss	Err. [m]	Cont.
baseline	0.000	0.000	1.000	—	0.000
isac_full	0.280	0.023	0.720	1.032	0.273
isac_no_mti	0.290	0.033	0.710	1.011	0.283
isac_narrow	0.290	0.033	0.710	1.012	0.283
isac_wide	0.290	0.022	0.710	1.012	0.283
isac_deep	0.290	0.121	0.710	1.129	0.283
isac_no_tracker	0.283	0.336	0.717	1.252	0.273

The simulation running on NR baseline configuration without ISAC components and NR ISAC configuration are evaluated for 2x2, 4x4, and 8x8 gNB array sizes. Both scenarios use the same warehouse layout, wireless node positions, robot mobility model, radio parameters, and application workload, so the comparison isolates the effect of enabling situation awareness in the Sionna-backed channel.

1) *Warehouse Digital Twin*: To evaluate the framework in a realistic 6G warehouse environment, we created a digital twin application for warehouse. The whole workflow is managed by the a warehouse controller. The controller communicates with package sensors, racks, temperature and humidity sensors, mobile robots, video clients, and cameras. The package sensors and racks are stationary, while the robots move between the package sensor area, the racks and dropoff area. The sensors report the status to the controller periodically. According to the status of the sensors, the controller sends commands to the robots to pickup the packages, put them on the rack or drop off at the dropoff area. The controller track the positions of the robots by communicating with the gNB which keeps the positions of the robots in the system state. The robots are equipped with cameras on them for monitoring the packages, the racks, and the environment and the user can request the video stream from the cameras through the video clients.

Among the warehouse devices, the package sensors, rack sensors, temperature and humidity sensors are stationary while the robots are mobile. The robots are main targets for the ISAC evaluation because they are mobile, the propagation conditions change frequently, and the robot positions are tracked by the gNB. Other wireless devices create background traffic and competing wireless flows. Therefore, this scenario tests whether ISAC can improve robot communication while keeping the rest of the warehouse workload active.

The performance is evaluated in terms of application-related KPIs, propagation-related KPIs, power consumption related KPIs and sensing related KPIs. First, application flows are grouped by node type to measure throughput, delay, jitter, and packet loss. Next, Sionna propagation traces are grouped by device type to explain the channel behavior. Then, energy traces are used to measure the power consumption of gNB according to the array size and ISAC activation. Finally, the ISAC detected-object output is used to evaluate robot detection, tracking continuity, and localization error.

2) *Application-related KPIs*: The application results in Fig. 4 are considered first because they show the end-to-end effect experienced by the warehouse traffic. At 2×2 , ISAC does not yet provide a stable system-level benefit: con-

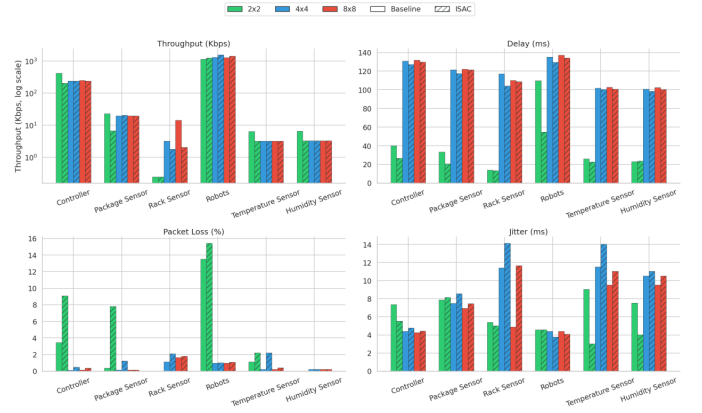


Fig. 4. Application-level KPIs under baseline and ISAC for the three gNB array sizes.

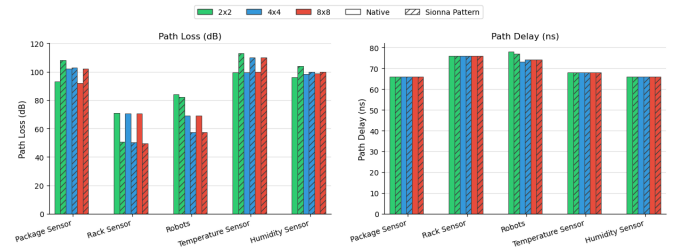


Fig. 5. Propagation KPIs grouped by wireless node category, with solid bars for Native and hatched bars for Sionna Pattern.

troller throughput drops from 417.674 kbps to 205.345 kbps, and package-sensor throughput drops from 22.990 kbps to 6.634 kbps. Robot throughput increases slightly from 1159.444 kbps to 1249.326 kbps, but this increase is accompanied by 15.439% packet loss, so the small-array ISAC case is better interpreted as unstable delivery rather than a useful gain.

When the gNB array is increased to 4×4 , the application behavior changes because the ISAC configuration gains enough spatial resolution to support the dominant mobile links. Controller and package-sensor throughput return to the baseline range, while robot throughput increases to 1582.081 kbps compared with the baseline value of 1328.452 kbps. The 8×8 ISAC case keeps the same robot-dominant trend but reduces robot throughput to 1445.378 kbps, so the larger array does not provide a proportional additional application gain. The application layer therefore identifies 4×4 as the first array size where ISAC becomes useful for the warehouse workload.

3) *Propagation-related KPIs*: The propagation results in Fig. 5 explain why the application-level gain appears only after the array reaches 4×4 . For robot links, baseline path loss decreases from 83.948 dB at 2×2 to approximately 69.2 dB at 4×4 and 8×8 . Under ISAC, the same trend is stronger: robot path loss falls from 82.318 dB at 2×2 to 57.359 dB at 4×4 and then remains almost unchanged at 8×8 . This channel-level saturation matches the application-level saturation, because most of the useful ISAC gain is already obtained at 4×4 .

The propagation improvement is not uniform across all warehouse devices, and this selectivity is important for inter-

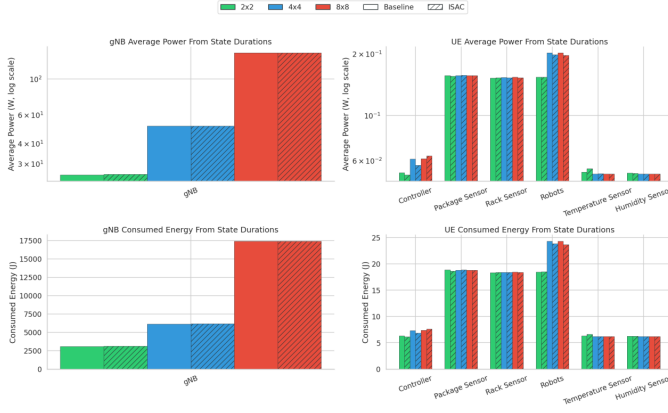


Fig. 6. Average power and consumed energy under baseline and ISAC.

preparing the system result. Rack-sensor links also benefit from ISAC, with roughly 20 dB lower path loss than the baseline across the tested array sizes. In contrast, package, temperature, and humidity sensors experience higher path loss under ISAC than under the baseline. The overall throughput improvement is therefore driven mainly by the mobile robot links, which are the links most directly aligned with the sensing-informed beamforming objective.

4) *Power Consumption related KPIs*: The power results in Fig. 6 are evaluated after the communication and propagation results because they determine whether the observed gains are practically affordable. The dominant power factor is the gNB array size rather than ISAC activation alone. The gNB average power rises from approximately 25.6 W at 2×2 to 51.3 W at 4×4 , and then to about 145 W at 8×8 . The corresponding gNB energy increases from about 3.1 kJ to 6.15 kJ and then to approximately 17.4 kJ.

This energy trend changes the interpretation of the larger array sizes. At the system level, total reported energy is nearly identical between baseline and ISAC for the 4×4 and 8×8 cases, while ISAC slightly improves the useful throughput. However, the 8×8 array requires much higher gNB power without giving a proportional communication gain over 4×4 . Therefore, the energy results reinforce the application and propagation results: 4×4 is the balanced operating point for the tested warehouse workload.

5) *Sensing and Localization*: The sensing results are evaluated last because they are available only in the ISAC scenario and because they explain whether the communication gains are supported by meaningful target awareness. Fig. 7 shows that the main sensing improvement occurs when the gNB array is increased from 2×2 to 4×4 . The detection rate rises from 0.485 to 0.523, and the missed-detection rate correspondingly falls from 0.515 to approximately 0.477. The 8×8 case changes these values only slightly, with a detection rate of 0.526, so sensing also saturates after the 4×4 configuration.

False positives and track continuity clarify the quality of those detections. Validated false positives are already low at 2×2 and fall to zero for both 4×4 and 8×8 . Track continuity remains close to unity across the three array sizes, which means that once a valid robot detection sequence



Fig. 7. Sensing KPIs for the ISAC warehouse scenario.

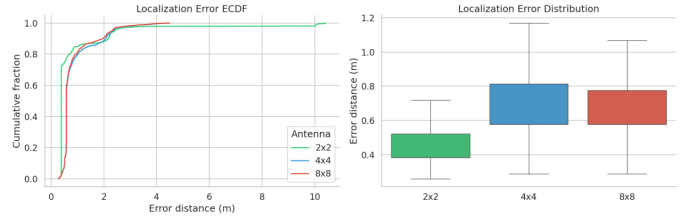


Fig. 8. Localization-error distribution for the ISAC warehouse scenario.

is formed, the exported identity remains stable. The larger arrays therefore improve sensing mostly by increasing valid detection opportunities and reducing unstable detections, not by fundamentally changing the tracking behavior.

The localization-error distribution in Fig. 8 shows the same stability effect from a different angle. The 2×2 configuration has the lowest median error at 0.384 m, but it also has the largest maximum error at 10.427 m. The 4×4 and 8×8 configurations have slightly higher median errors of about 0.577 m, but their maximum errors are reduced to around 4.5 m and their upper-tail errors are lower. Thus, the larger arrays do not mainly improve pointwise median localization; instead, they suppress large-error outliers and make the sensing output more stable.

Overall, the warehouse use-case experiment shows that ISAC benefit depends on sufficient antenna-array support. The 2×2 ISAC configuration is not stable enough for the mixed warehouse workload, whereas the 4×4 configuration improves the dominant robot communication links, improves sensing stability, and avoids the much higher gNB power cost of 8×8 . The results therefore support 4×4 as the most balanced configuration among the tested cases.

III. CONCLUSION AND FUTURE WORK

This paper presented a situation-aware ISAC co-simulation framework for 6G smart warehouses. The framework connects Sionna RT with ns-3/5G LENA so that site-specific ray tracing, sensing, tracking, adaptive beamforming, and NR communication can be evaluated in one workflow. The sensing module converts ray-traced reflections into target detections, the tracking module stabilizes those detections over time, and

the beamforming module uses the confirmed target positions to update the communication link.

The controlled validation shows that the framework behaves as expected when the scene becomes more difficult. In free space, visibility is favorable, so ISAC reduces the mean path loss by up to about 18 dB and keeps the false-positive rate low when tracking is enabled. In the warehouse scene, ISAC still improves propagation, with the widest tested beam giving the strongest path-loss reduction. However, racks, walls, and partial visibility reduce the detection rate and increase missed detections. This means that the main warehouse sensing problem is not pointwise localization accuracy after a target is detected, but maintaining complete and continuous detections in clutter. The ablation results also show two important trade-offs: deeper propagation gives only a small detection gain and can add false positives, while removing the tracker increases raw detections but makes the output much less clean.

The end-to-end warehouse Digital Twin confirms that these controlled findings matter at application level. The 2×2 gNB array is not sufficient for a stable ISAC gain because robot traffic becomes less reliable. The 4×4 array gives the best operating point: it improves robot-link propagation and throughput, keeps the rest of the warehouse traffic usable, improves sensing stability, and avoids the large power increase of the 8×8 array. The 8×8 case provides more antenna resources, but its extra energy cost is not matched by a proportional communication or sensing improvement.

These results show that ISAC should not be evaluated from sensing accuracy or communication throughput alone. A useful warehouse deployment must balance all three dimensions: reliable target awareness, useful link improvement, and reasonable energy cost. Future work should therefore focus on more robust clutter handling, stronger track continuity in dense indoor scenes, and beam-management methods that combine target position with channel state information. The evaluation should also be extended with more traffic loads, random seeds, robot routes, and warehouse layouts so that the operating limits of the framework can be characterized more completely.

REFERENCES

- [1] H. Wymeersch *et al.*, “Positioning and sensing in 6G: Gaps, challenges, and opportunities,” *IEEE Vehicular Technology Magazine*, 2022.
- [2] Y. Heng *et al.*, “AI-enabled multimodal beamforming for next-generation wireless systems: A survey,” *IEEE Communications Surveys & Tutorials*, 2023.
- [3] Authors, “Memory-full context-aware predictive mobility management in dual-connectivity 5G networks,” *IEEE Trans. on Mobile Computing*.
- [4] J. Hoydis *et al.*, “Sionna RT: Differentiable ray tracing for radio propagation modeling,” NVIDIA, 2023. <https://nvlabs.github.io/sionna/>.
- [5] A. Authors, “Ns3Sionna: Integrating Sionna RT GPU-accelerated ray tracing into ns-3,” in *Proc. Workshop on ns-3 (WNS3)*, 2024.
- [6] N. Patriciello *et al.*, “An E2E simulator for 5G NR networks,” *Simulation Modelling Practice and Theory*, vol. 96, 2019.
- [7] 3GPP, “Study on channel model for frequencies from 0.5 to 100 GHz,” TR 38.901, 3rd Generation Partnership Project.
- [8] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proc. KDD*, 1996, pp. 226–231.
- [9] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [10] G. Nardini *et al.*, “Simu5G—an OMNeT++ library for end-to-end performance evaluation of 5G networks,” *IEEE Access*, vol. 8, 2020.